AI-DRIVEN ADAPTIVE CRYPTOGRAPHIC POLICY ENGINE FOR SECURE DATA SHARING

Project Report
submitted in partial fulfilment for the degree of
Master of Technology
Under the
Faculty of Engineering

Submitted by

BARNABA BOBBILI

CB.SC.P2CSE25002

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMRITA SCHOOL OF COMPUTING,

AMRITA VISHWA VIDYAPEETHAM,

February, 2026

COIMATORE CAMPUS

Contents

1 Abstract **1**

2 Introduction **1**

3 Problem Statement **1**

4 Project Objectives **2**

5 Project Modules **2**

5.1 Backend Module (FastAPI) 2

5.2 Frontend Module (React + TypeScript) 3

5.3 Security and Governance Module 3

5.4 Data, Messaging, and Analytics Module 3

6 Azure Services Used **3**

7 Project Execution Steps **4**

7.1 Step 1: Foundation and Identity Setup 5

7.2 Step 2: Data and Security Baseline 5

7.3 Step 3: Backend Container Deployment 5

7.4 Step 4: Frontend Deployment 5

7.5 Step 5: Integration Services Enablement 5

7.6 Step 6: Data Migration and Validation 6

8 Results and Discussion **6**

9 Conclusion and Future Work **6**

1 Abstract

Weaver is an AI-driven adaptive cryptographic policy engine designed to secure data sharing workflows by combining sensitivity-aware classification with policy-enforced encryption controls. The project addresses a practical gap in enterprise systems where security enforcement is often static, while data risk and user context are dynamic. The implementation provides a full-stack platform with authentication, multi-factor authentication (MFA), role-based access control, classification, encryption/decryption, secure sharing, administration, and analytics support. The solution is deployed on Azure using managed services for compute, data, secrets, messaging, machine learning, observability, and analytics. This report presents the problem context, project objectives, module-level architecture, cloud service mapping, and detailed project execution steps.

2 Introduction

Digital systems increasingly process mixed-sensitivity data, including public records, internal business information, confidential documents, and highly sensitive payloads. Applying one fixed security configuration to all categories leads to either over-protection (high cost and friction) or under-protection (high risk). Weaver was built to operationalize adaptive cryptographic enforcement where security controls are selected based on data sensitivity and governance policy.

The project aligns security engineering, cloud-native architecture, and applied machine learning. It is designed to be practical for real application pipelines where users, data, and policy constraints change continuously.

3 Problem Statement

The central problem solved by Weaver is:

> How can a data sharing platform automatically and reliably apply the correct cryptographic protection level for each payload while preserving usability, enforceability, and auditability?

Key pain points in conventional systems:

1. Uniform security policy for heterogeneous data risk.
2. Weak coupling between identity posture (for example MFA status) and cryptographic action authorization.

3. Limited audit visibility for security-critical operations.
4. Fragmented analytics for operational and compliance decisions.

4 Project Objectives

The project objectives are:

1. Build a secure end-to-end data handling platform for classification, encryption/decryption, and sharing.
2. Implement policy-driven cryptographic adaptation based on sensitivity classes.
3. Enforce role-based access and MFA-aware controls for high-risk operations.
4. Deploy the platform in a cloud-native architecture on Azure with managed services.
5. Provide observability, audit trails, and analytics integration suitable for governance workflows.

5 Project Modules

5.1 Backend Module (FastAPI)

The backend implements business logic and security enforcement:

- Authentication, authorization, and token lifecycle management.
- MFA setup, verification, disable, and recovery-code flows.
- Sensitivity classification endpoints for text and file workflows.
- Policy-based encryption/decryption services.
- Secure sharing and guest retrieval endpoints.
- Administrative APIs for user, policy, and compliance operations.
- Health, readiness, and liveness endpoints for operational reliability.

5.2 Frontend Module (React + TypeScript)

The frontend provides role-aware user interaction:

- Public pages: landing, login, registration, and token-based decrypt view.
- Authenticated pages: dashboard, classify, history, shares, profile, and help.
- Admin pages: users, policies, shares, compliance, and audit-oriented views.
- Analyst pages: analytics and benchmark-oriented workflows.

5.3 Security and Governance Module

Security controls include:

- RBAC with `admin`, `analyst`, and `viewer` roles.
- MFA (TOTP-based) for elevated protection paths.
- Rate limiting, session timeout middleware, and security headers.
- Audit and notification pipeline for critical events.

5.4 Data, Messaging, and Analytics Module

- Azure PostgreSQL for transactional persistence.
- Blob storage for large encrypted payloads.
- Service Bus queues for asynchronous processing and decoupling.
- Synapse export path for analytical workloads and Power BI integration.

6 Azure Services Used

The production environment in resource group `weaver-rg` uses the services below.

Category	Azure Service	Resource Name(s)	Purpose
Compute	Container Apps	`weaver-env`, `weaver-backend`	Scalable backend runtime

Category	Azure Service	Resource Name(s)	Purpose
Frontend	Static Web Apps	`weaver-frontend`	Production web UI hosting
Database	PostgreSQL Flexible Server	`weaver-db-prod`	Primary OLTP data store
Secrets	Key Vault	`weaver-kv-ijbkmp25`	Secret management and retrieval
Identity	Managed Identity	`weaver-backend-identity`	Service-to-service authentication
Storage	Storage Account	`weaverstorageprod`	Encrypted object/blob storage
Messaging	Service Bus	`weaver-servicebus-prod`	Async queue-based processing
Monitoring	Application Insights	`weaver-appinsights`	Telemetry and diagnostics
API Governance	API Management	`weaver-apim`	Managed API gateway layer
Machine Learning	Azure Machine Learning	`weaver-ml-workspace`	Model management and online endpoints
Analytics	Azure Synapse	`weaver-synapse-ws`	Analytics query and warehouse path
BI Capacity	Power BI Dedicated	`weaveranalytics`	Dedicated BI compute capacity

Operational Endpoints

- Frontend: https://salmon-meadow-04fa55300.1.azurestaticapps.net
- Backend: https://weaver-backend.whitehill-eea76820.centralindia.azurecontainerapps.io
- API Gateway: https://weaver-apim.azure-api.net
- PostgreSQL FQDN: `weaver-db-prod.postgres.database.azure.com`
- Synapse serverless SQL (Power BI server value): `weaver-synapse-ws-ondemand.sql.azuresynapse`

7 Project Execution Steps

The implementation was executed through the following steps.

7.1 Step 1: Foundation and Identity Setup

1. Create the Azure resource group and register required providers.
2. Provision user-assigned managed identity for backend workloads.
3. Configure role assignments for Key Vault, storage, and dependent services.

7.2 Step 2: Data and Security Baseline

1. Provision Key Vault and initialize required secret keys.
2. Provision PostgreSQL Flexible Server and application database.
3. Configure secure connectivity, firewall rules, and SSL-required DB access.
4. Provision storage containers for encrypted payload artifacts.

7.3 Step 3: Backend Container Deployment

1. Build and publish backend container image.
2. Deploy backend application to Azure Container Apps.
3. Inject runtime bootstrap configuration (identity, Key Vault URL, CORS).
4. Validate backend availability using health endpoints.

7.4 Step 4: Frontend Deployment

1. Build frontend assets with production backend endpoint.
2. Deploy to Azure Static Web Apps.
3. Validate route handling and backend API connectivity.

7.5 Step 5: Integration Services Enablement

1. Configure Service Bus queue channels for asynchronous workflows.
2. Enable Application Insights telemetry and diagnostics.
3. Configure API Management gateway policies.
4. Configure Azure ML workspace and online endpoint integration.
5. Configure Synapse integration path for analytics export.

7.6 Step 6: Data Migration and Validation

1. Export source data from Supabase direct PostgreSQL connection.
2. Import into Azure PostgreSQL using compatibility-safe restore procedure.
3. Verify schema availability and table-level record consistency.
4. Execute user workflow checks for authentication, MFA, classification, encryption, and administration.

8 Results and Discussion

The deployed platform satisfies the primary engineering goals:

- Adaptive cryptographic logic is operational across key business flows.
- Security controls (RBAC and MFA) are integrated with runtime actions.
- Cloud architecture provides modular scalability and managed operations.
- Monitoring and analytics integration support operational visibility.

The implementation also highlights practical cloud engineering considerations:

- Service naming consistency across scripts and runtime variables is critical.
- Data migration between PostgreSQL variants may require compatibility filtering.
- Academic and operational documentation should be maintained as versioned artifacts.

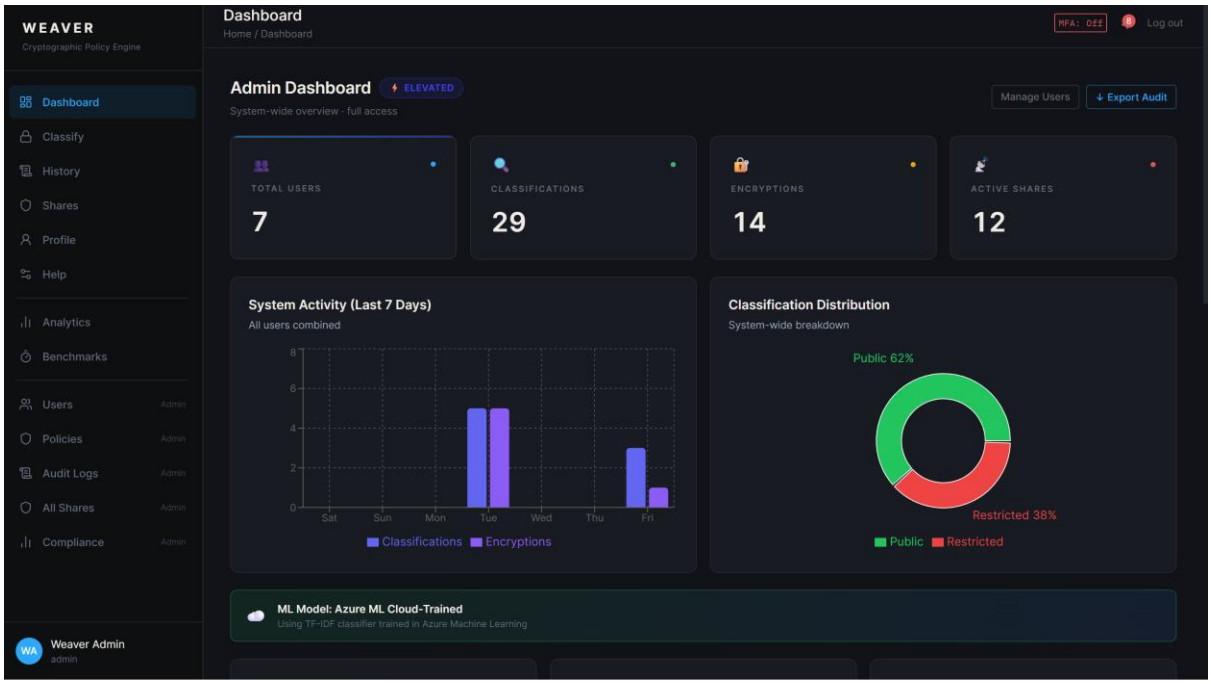
9 Conclusion and Future Work

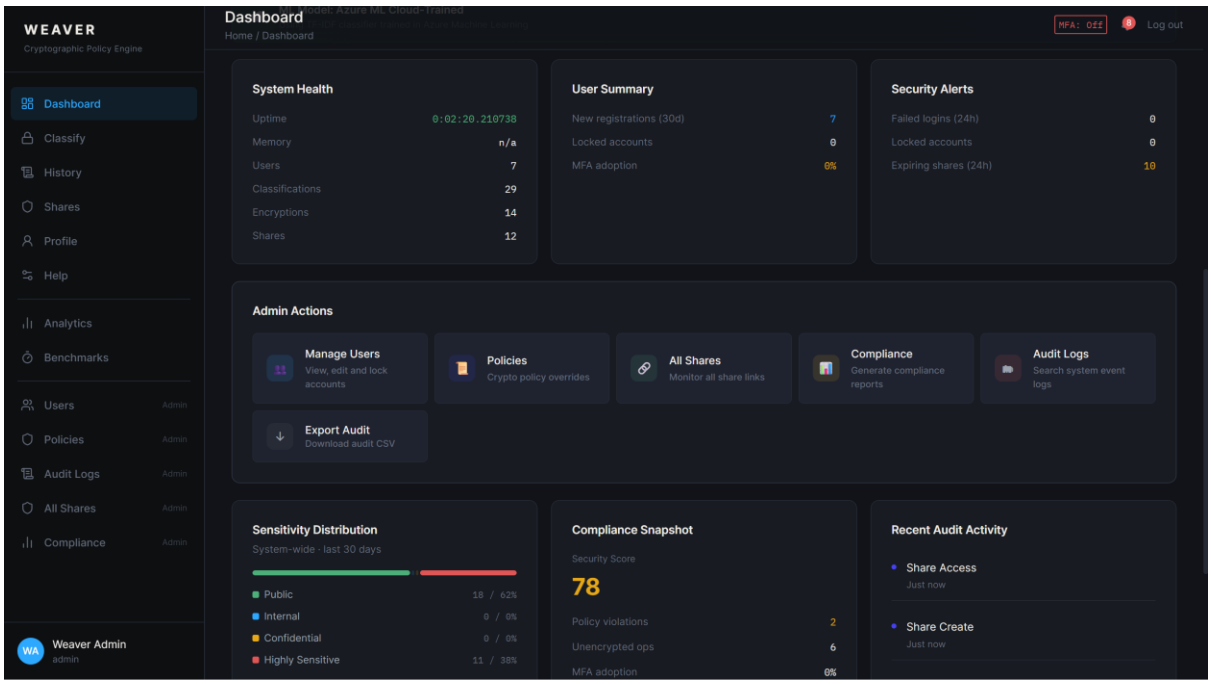
Weaver demonstrates a viable cloud-native model for adaptive cryptographic policy enforcement. The system combines application security engineering, policy control, and cloud service integration to produce a practical secure data-sharing platform.

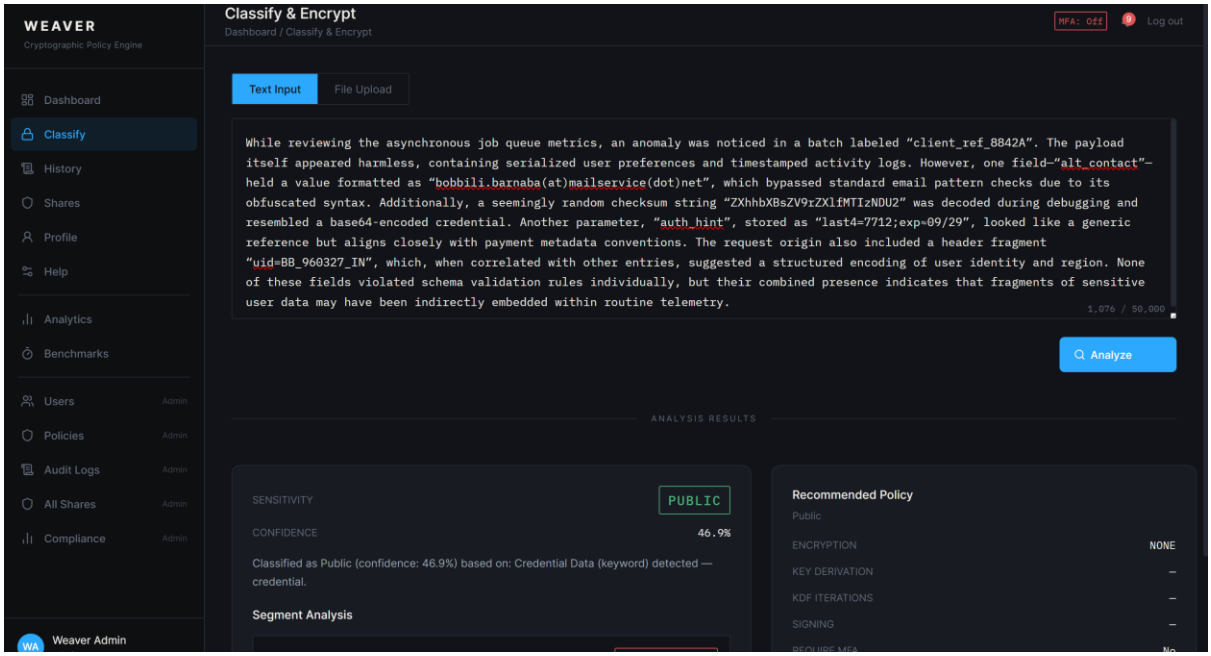
Future improvements include:

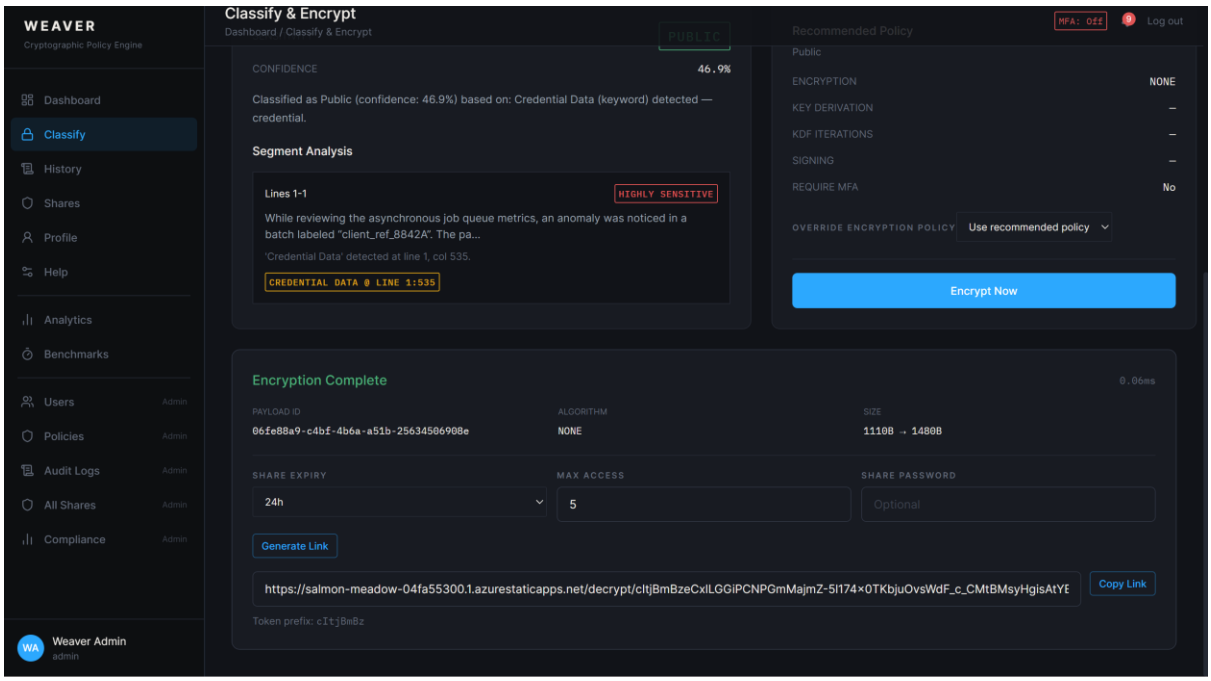
- Expanded automated integration tests across cloud dependencies.
- Additional compliance controls and reporting templates.
- Enhanced network hardening through private endpoints and tighter segmentation.
- Model drift monitoring and periodic ML retraining governance.

Screenshots






WEAVER
Cryptographic Policy Engine
Dashboard
Classify
History
Shares
Profile
Help
Analytics
Benchmarks
Users
Policies
Audit Logs
All Shares
Compliance
Weaver Admin
Classify & Encrypt
Dashboard / Classify & Encrypt
MFA: Off
Log out
Text Input
File Upload
While reviewing the asynchronous job queue metrics, an anomaly was noticed in a batch labeled "client_ref_8842A". The payload itself appeared harmless, containing serialized user preferences and timestamped activity logs. However, one field—"alt_contact"—held a value formatted as "bobbili.barnaba(at)mailservice(dot)net", which bypassed standard email pattern checks due to its obfuscated syntax. Additionally, a seemingly random checksum string "ZXhhbXBsZV9rZXlfMTIzNDU2" was decoded during debugging and resembled a base64-encoded credential. Another parameter, "auth_hint", stored as "last4=7712;exp=09/29", looked like a generic reference but aligns closely with payment metadata conventions. The request origin also included a header fragment "uid=BB_960327_IN", which, when correlated with other entries, suggested a structured encoding of user identity and region. None of these fields violated schema validation rules individually, but their combined presence indicates that fragments of sensitive user data may have been indirectly embedded within routine telemetry.
1,076 / 50,000
Analyze
ANALYSIS RESULTS
SENSITIVITY
PUBLIC
CONFIDENCE
46.9%
Classified as Public (confidence: 46.9%) based on: Credential Data (keyword) detected — credential.
Segment Analysis
Recommended Policy
Public
ENCRYPTION
NONE
KEY DERIVATION
KDF ITERATIONS
SIGNING


WEAVER
Cryptographic Policy Engine
Dashboard
Classify
History
Shares
Profile
Help
Analytics
Benchmarks
Users
Policies
Audit Logs
All Shares
Compliance
Weaver Admin
admin
Classify & Encrypt
Dashboard / Classify & Encrypt
MFA: Off
Log out
CONFIDENCE
46.9%
Classified as Public (confidence: 46.9%) based on: Credential Data (keyword) detected — credential.
Segment Analysis
Lines 1-1
HIGHLY SENSITIVE
While reviewing the asynchronous job queue metrics, an anomaly was noticed in a batch labeled "client_ref_8842A". The pa...
'Credential Data' detected at line 1, col 535.
CREDENTIAL DATA @ LINE 1:535
Recommended Policy
Public
ENCRYPTION
NONE
KEY DERIVATION
KDF ITERATIONS
SIGNING
REQUIRE MFA
No
OVERRIDE ENCRYPTION POLICY
Use recommended policy
Encrypt Now
Encryption Complete
0.06ms
PAYLOAD ID
06fe88a9-c4bf-4b6a-a51b-25634506908e
ALGORITHM
NONE
SIZE
1110B → 1480B
SHARE EXPIRY
24h
MAX ACCESS
5
SHARE PASSWORD
Optional
Generate Link
https://salmon-meadow-04fa55300.1.azurestaticapps.net/decrypt/cItjBmBzeCxILGGiPCNPGmMajmZ-5I174×0TKbjuOvsWdF_c_CMtBMsyHgisAtYE
Copy Link
Token prefix: cItjBmBz